

Communication Skill for Software and Knowledge Engineers

Lec. #05

Research Presentation

Class Schedule

#	Date	Content
01	08 / Aug / 2025	- Course Introduction (Lecture) - How to Give a Self-Introduction (Lecture)
02	15 / Aug / 2025	- Self-Introduction Presentation (Workshop) - How to Explain a Software Library (Lecture)
03	22 / Aug / 2025	- Software Library Presentation (Workshop) - How to Share Team Status Updates (Lecture)
04	29 / Aug / 2025	- Sprint Review Presentation (Workshop) - How to Turn a Research Paper to Presentation (Lecture)
05	05 / Sep / 2025	- Research Presentation (Workshop) - How to Pitch a Product (Lecture)
06	12 / Sep / 2025	- Pitching (Workshop) - How to Deliver a TED-Style Talk (Lecture)
07	19 / Sep / 2025	- TED-Style Talk Presentations (Workshop)
(20 / Sep / 2025 – 28 / Sep / 2025) Midterm Examination		

How to pitch a product

Why Pitching is Matter

Why Pitching is Matter

Engineers are builders — but often not storytellers.

Many engineers develop technically sound solutions that never gain traction.

Why?

- ✗ **Too much technical detail** without context
- ✗ **Missing the “so what?”** — no clear value proposition
- ✗ **Poor alignment** with business goals or user needs

Truth: Great tech doesn't sell itself — **clear communication does.**

The Reality in Tech Environments

Stakeholders (execs, PMs, clients) don't evaluate **code quality**

They evaluate:

- ✓ *Does it solve a real problem?*
- ✓ *Will it save time or money?*
- ✓ *Is it better than the current approach?*

Communication is often the **difference between a greenlight and a dead end**, no matter how brilliant the solution.

Know Your Audience

“If you speak to everyone the same way, you're not speaking to anyone effectively.”

The **same product** should be presented **differently** depending on who's listening.

Failing to tailor your pitch often leads to:

- ✓ Glossed-over eyes from execs
- ✓ Misalignment with PMs
- ✓ Mistrust or confusion from engineers
- ✓ Indifference from customers

Different Audiences, Different Priorities

	 Audience	 They Care About	 Your Pitch Should Emphasize
	Executives	ROI, growth, strategic differentiation	Business value, KPIs, competitive edge, cost savings
	Product Managers	Timelines, user needs, roadmap fit	Feature scope, feasibility, priority alignment
	Developers/Engineers	Architecture, scalability, API design	Tech stack, modularity, code quality, integration ease
	Customers/Clients	Simplicity, outcomes, problem-solving	Benefits, usability, real-world impact, onboarding ease
	Designers (optional)	Workflow, consistency, UX patterns	UI flexibility, handoff clarity, accessibility support

Different Audiences, Different Priorities

 Audience	
 Executives	“This tool could reduce production incidents by 35%, saving ~\$300K/year.”
 Product Managers	“This gives early visibility into potential delays and improves sprint planning.”
 Developers/Engineers	“It integrates with GitHub Actions and flags risky PRs using a trained model.”
 Customers/Clients	“It helps your team catch bugs earlier, so users face fewer crashes.”

Example: AI-based bug prediction tool

One Product = Many Perspective



When preparing a pitch:

- Always **identify who's in the room**
- For mixed audiences, **start broad → dive deeper** depending on questions
- Create **multiple versions** of your pitch — even if it's the same solution

Crafting the Core Narrative

"Facts tell, but stories sell."

"Facts tell, but stories sell."

Stakeholders rarely buy into **raw specs or features**.
What they remember — and act on — **is a compelling narrative**.

A great pitch tells a story with:

- ☐ Tension (a problem worth solving)
- ☐ A hero (your product)
- ☐ Stakes (why it matters *now*)
- ☐ Uniqueness (why your solution stands out)

The 4-Part Structure

1. Problem

2. Solution

3. Why Now?

4. Differentiator

The 4-Part Structure

1. Problem

What pain are users or the business feeling?

- Quantify if possible: time wasted, money lost, frustration
- Make it relatable and concrete
- Focus on *why* it's urgent or expensive to ignore

“Syncing customer records across tools is manual, error-prone, and eats up 6+ hours per week.”

2.

3.

4.

The 4-Part Structure

2. Solution

What have you built to solve that problem?

- Be clear, simple, and benefit-driven
- Emphasize outcomes, not implementation details
- Use simple analogies if needed

“We built a tool that connects Salesforce and Jira in one click — no code, no sync issues.”

3.

4.

The 4-Part Structure

3. Why Now?

Why is this problem urgent **today**?

- Highlight trends, recent shifts, or new opportunities
- This creates **momentum** and **relevance**

“With remote teams growing, data fragmentation is at an all-time high — this pain is scaling fast.”

4.

The 4-Part Structure

3. Why Now?



4.

The 4-Part Structure

3. Why Now?



4.

4. Differentiator

Why is your approach better than others?

- What's **unique** about your tech, method, or experience?
- Avoid trash-talking competitors — focus on *your edge*

“Unlike Zapier or custom scripts, ours requires no setup, handles schema drift, and works in real time.”

Use Anchors

➤ Numbers:

- “2x faster,” “\$50K annual savings,” “<30 min setup”

➤ Analogies:

- "Think Calendly, but for cross-tool data sync"

➤ Visuals:

- Before/after diagrams, side-by-side comparisons

Example

“Manual data syncing wastes 6 hours per week per engineer. Our tool automates that with a 1-click setup, saving time and preventing data drift. As remote teams and SaaS tool usage skyrocket, this pain is only growing. And unlike other solutions, we don’t require complex workflows or maintenance just connect and go.”

Structuring the Presentation

Why Structure Matters

“A confused audience never buys in structure creates clarity and momentum.”

A clear structure:

- Keeps the audience engaged
- Builds logical flow from **problem** → **solution** → **value**
- Helps technical and non-technical stakeholders follow along

Think of your presentation as **a guided tour**, not a feature dump.

Recommended Slide Flow

Each slide should cover **one main idea only**. No walls of text. Use visuals.

1. Title Slide
2. Problem Slide
3. Solution Slide
4. Demo or Product Flow
5. Value Slide
6. Differentiation Slide
7. Roadmap / CTA Slide

1. Title Slide

➤ Name of the product or feature

➤ Tagline that hints at the core benefit

Example:

“PulseSync: Real-time health tracking for frontline teams”

➤ Optional: Presenter name, team, and logo

2.
3.
4.
5.
6.
7.

2. Problem Slide

➤ Describe **what's broken** or inefficient

➤ Use:

 **Metrics:** “Teams lose 20% of time to status misalignment”

 **Stories:** “Sarah, a team lead, manually emails updates daily...”

 **Quotes:** “It feels like we’re always out of sync”

3.

4.

5.

6.

7.

3. Solution Slide

- One powerful sentence:
“PulseSync gives managers a real-time dashboard of field activity.”
- Show a **high-level graphic or diagram** (e.g., a workflow, interface mockup)
- Keep it **simple and benefit-focused**

4.
5.
6.
7.

4. Demo or Product Flow

- Optional live demo or video
- Or use **3-5 screenshots or steps** to explain:
 - User logs in
 - Connects to tools
 - Gets auto-generated summary
- Use callouts or animations to **highlight key steps**

5.
6.
7.

5. Value Slide

➤ Focus on **impact**, not just functionality:



Time saved



Cost reduction



Risk avoided



User satisfaction

➤ Format:

- Bullet list of 3 benefits (each with metric or outcome) 6.
- Optional visual: before/after table, ROI chart 7.

6. Differentiation Slide

- Explain what makes your solution **stand out**
 - Unique architecture?
 - Speed or simplicity?
 - UX? AI? Automation?
- Use side-by-side comparisons or bold claims like:
“ *4x faster than Zapier — with zero setup time.*”

7.

7. Roadmap / CTA Slide

➤ End with **momentum and clarity**:



What's next?

(features, scale, launch timeline)



What do you want from the audience?

(Feedback, funding, pilot users, etc.)

➤ Format:

- Visual roadmap (timeline or feature waves)
- Direct CTA: *“We’re looking for beta testers. Join us at...”*

Before/After Slide Review (Live Activity)

Show an **example of a cluttered slide**

(e.g., too many bullet points, low-contrast text, or jargon)

Then show a **cleaned-up version** with:

- Visual focus
- Fewer words
- Clear takeaway



Ask:

“What changed?”

“How does it feel now?”

“Which version would you trust more?”

Key Slide Design Tips

- Stick to **one message per slide**
- Use **icons, whitespace**, and minimal text
- Stick to 24pt+ font sizes
- Use consistent visual style (color, layout, font)

Designing Effective Slides & Visuals

“Your slides are not your script — they are your stage.”

In product pitches, **slides should support your story**, not compete with it.

Poor slide design causes:

- Information overload
- Lost audience attention
- Missed impact

Your goal: **clarity, focus, and visual persuasion**

Key Slide Design Principles

1. Show, Don't Tell
2. Use Simple Architecture Diagrams
3. Use Timelines and User Flows
4. Highlight Metrics and Before/After States
5. Use Code Sparingly

1. Show, Don't Tell

➤ Replace bullets with:

- Diagrams
- Icons
- Charts
- Annotated screenshots

➤ Visuals are processed **60,000x faster** than text



Instead of:

"We support multi-platform sync, low-latency updates, and admin controls"



Try:

A 3-icon horizontal layout:



Mobile sync



Real-time



Admin-safe

2.

3.

4.

5.

2. Use Simple Architecture Diagrams

- Keep it high-level — 3–5 components max
- Use arrows and color to show flow
- Label inputs, processing, and outputs

Example:

“Sync Flow: CRM → Webhook → DB Cache → UI Render”

3.

4.

5.

3. Use Timelines and User Flows

➤ Show progress visually:



Step-by-step: “How onboarding works”



Roadmaps: “Q2 → Q3 → Q4”

➤ Don’t crowd these — space is your friend

4.

5.

4. Highlight Metrics and Before/After States

➤ Show the **impact**:



“Before: Manual syncing — 6 hours/week”



“After: Automated — 10 minutes/month”

➤ Use:

- Side-by-side tables
- Bar graphs or % reductions
- Color coding (red → green)

5. Use Code Sparingly

- Only if your audience is **technical**
- Highlight the outcome, not the syntax

Instead of:

```
import requests
response = requests.get("https://api.example.com/data")
print(response.status_code)
print(response.json())
```

Say:

“With one API call, you get all active customers in <100ms.”



Tip: Add a small **caption** or callout like

“API returns JSON with user status, last activity, and flags.”

Design Basics

Element	Recommendation
Font size	Min 24pt (headlines 32pt+)
Color	Use 1–2 base colors + 1 highlight
Consistency	Stick to 1 font and layout system
Whitespace	Don't fear empty space
Contrast	Dark text on light background (or vice versa)

 Avoid:

- ✓ Red/green combos (colorblindness)
- ✓ Dense paragraphs
- ✓ Clipart or inconsistent icons

Design Basics

Good Slide Design = Clear Thinking

- ❖ Every slide should answer: **“What do I want them to remember?”**
- ❖ Use visuals, not explanations
- ❖ Focus on **benefit and clarity**, not decoration

Handling Questions & Objections

Types of Common Questions & Objections

Type	Example Question	Hidden Concern
Comparison	“How is this different from [existing tool]?”	Redundancy, feature overlap
Scalability/Tech Risk	“Will this work at our scale?”	Reliability, architecture limits
Cost or ROI	“What will it cost us — in money or effort?”	Budget, time investment
Adoption/Integration	“Will it work with our stack / existing tools?”	Compatibility, team disruption
Outcome skepticism	“What if no one uses it?”	Risk of low impact
Security/Compliance	“How secure is the data transfer?”	Risk, regulation, legal concerns

Handling Questions & Objections

In a product pitch — especially to **executives, product managers, or skeptical stakeholders** — questions are not just interruptions:

- They reveal **concerns, priorities, or misunderstandings**
- A well-handled question can **win buy-in**
- A poorly handled one can **undermine trust**



Great engineers don't just answer - they reframe and persuade.

Answering Framework

Use the **C-A-R** method:

Clarify → Address → Reconnect

❖ **Clarify** the question (if needed)

“Just to confirm — you're asking whether it handles 100k+ records in real-time?”

❖ **Address** the concern head-on

“Yes. We've tested it with 250k records and it sustained 45ms response time.”

❖ **Reconnect** to the benefit

“So teams save hours per week even at full load — and without touching infra.”

Answering Framework

If the concern is valid but solvable, say:

“You're right — that's a current limitation. But we have a roadmap item **next quarter** to address it with async caching.”

When You Don't Know the Answer

Be honest — never bluff.

“That’s a great question, and I don’t have the exact answer right now. I’ll follow up after checking with [team/data].”



Write it down: follow up via email or Slack.

Video



Q&A

Email: kundjanasith.th@ku.th